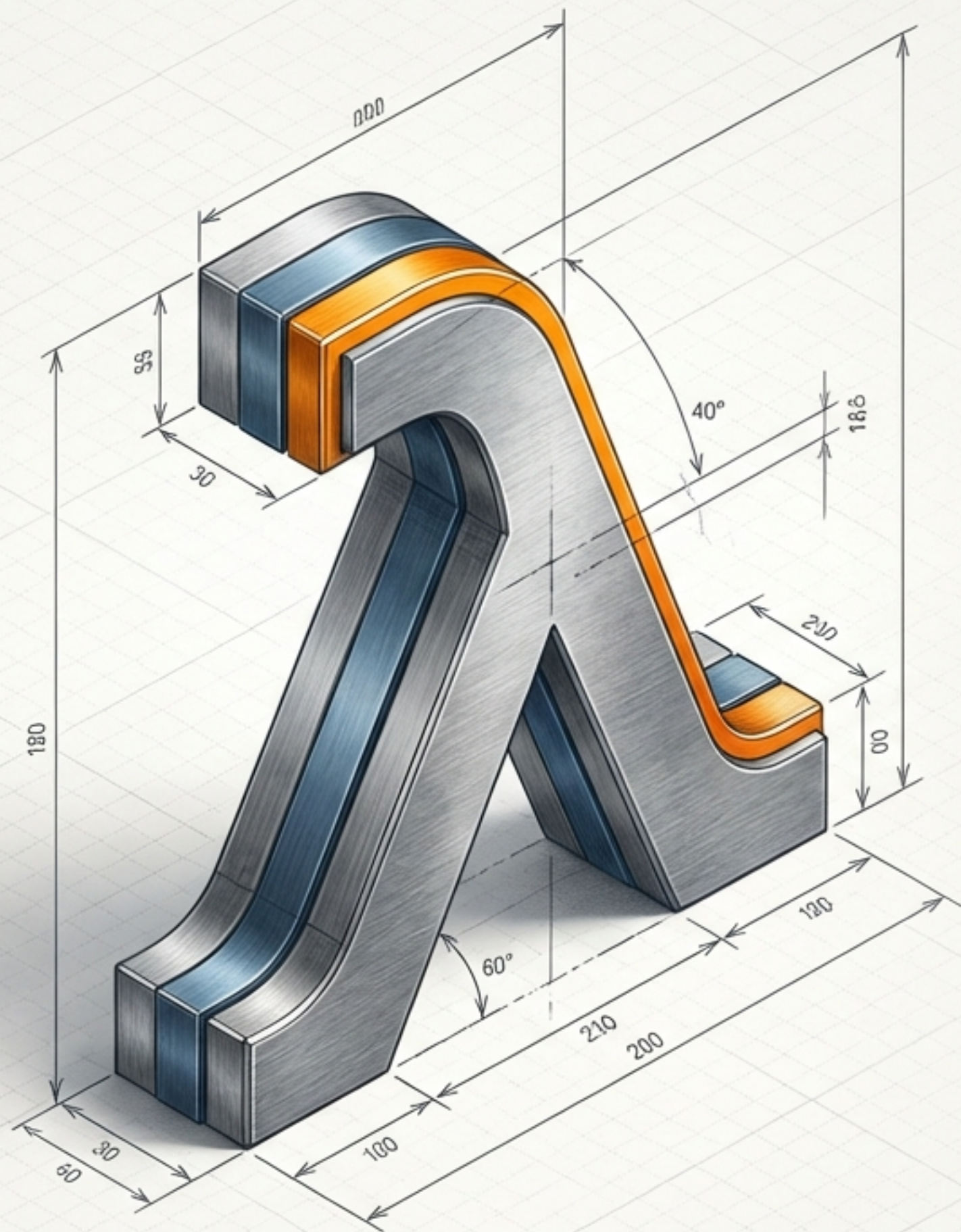


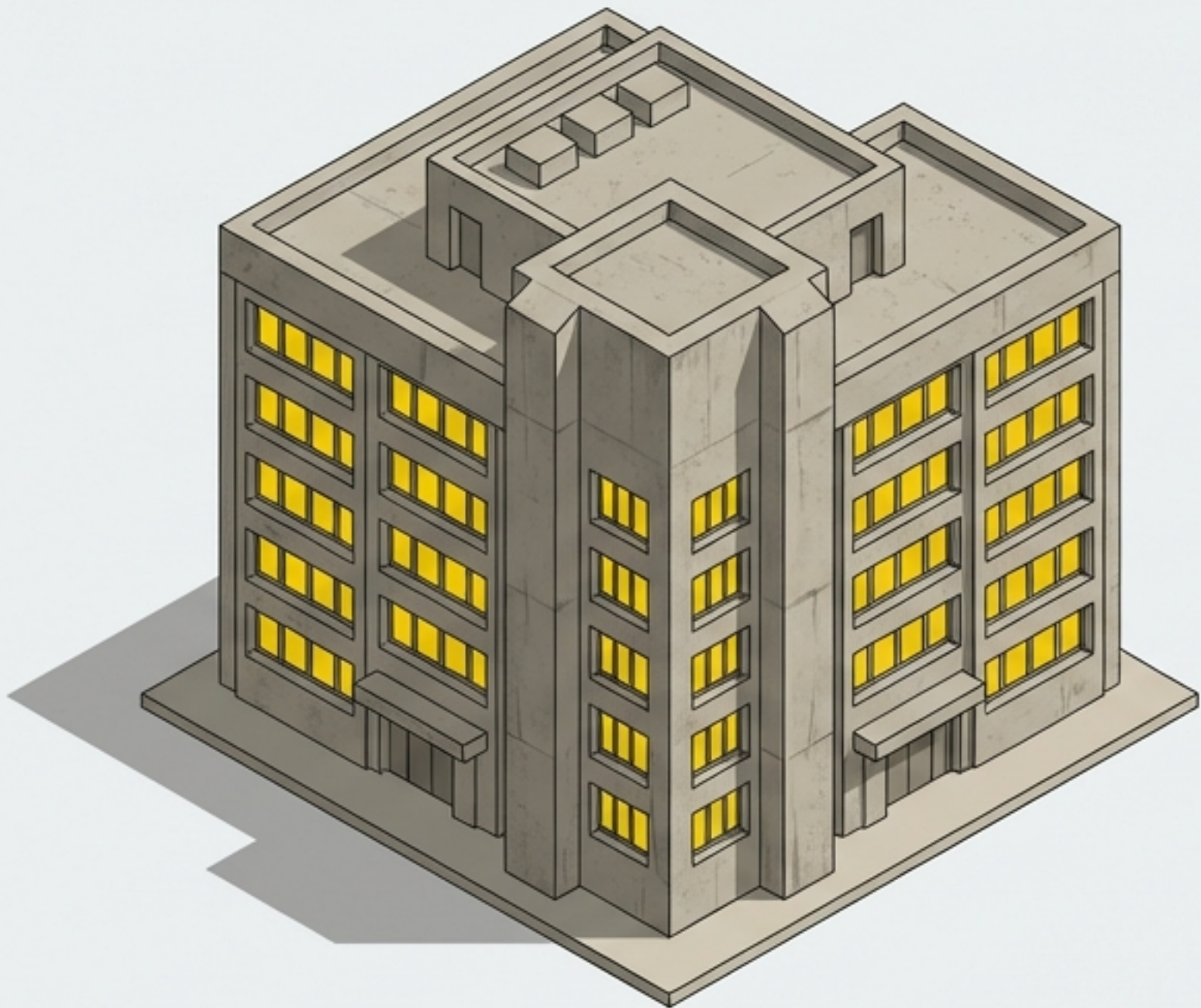


สถาปัตยกรรม Serverless สู่ความยืดหยุ่นที่ไร้ขีดจำกัด

คู่มือฉบับสมบูรณ์สำหรับผู้บริหารและนักพัฒนา

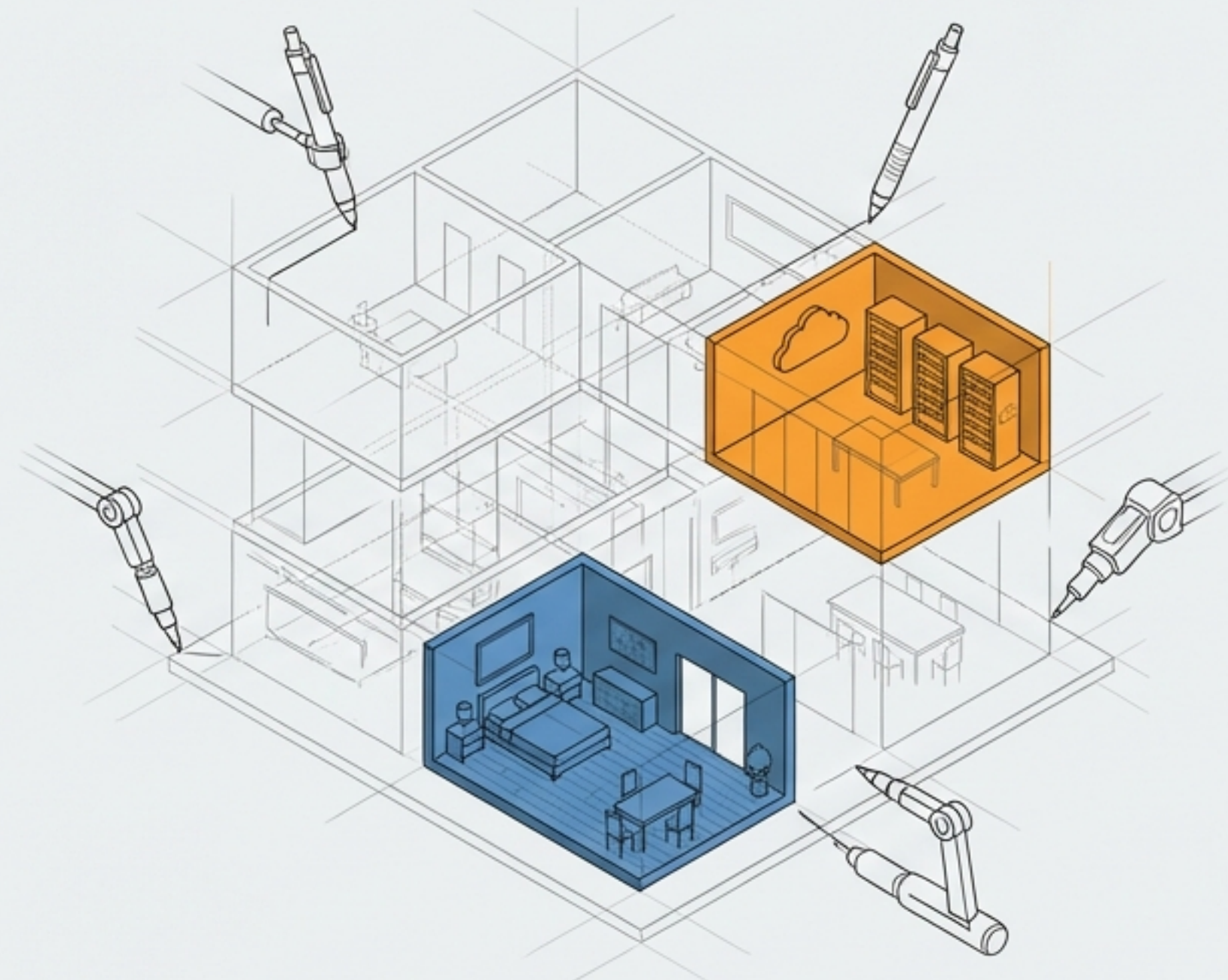
ทำความเข้าใจกลไก เศรษฐศาสตร์ และการควบคุม
AWS Lambda อย่างทะลุปรุโปร่ง

จากการซื้อบ้านสำเร็จรูป สู่การสร้างกลไกตามพิมพ์เขียว



Server-based Architecture

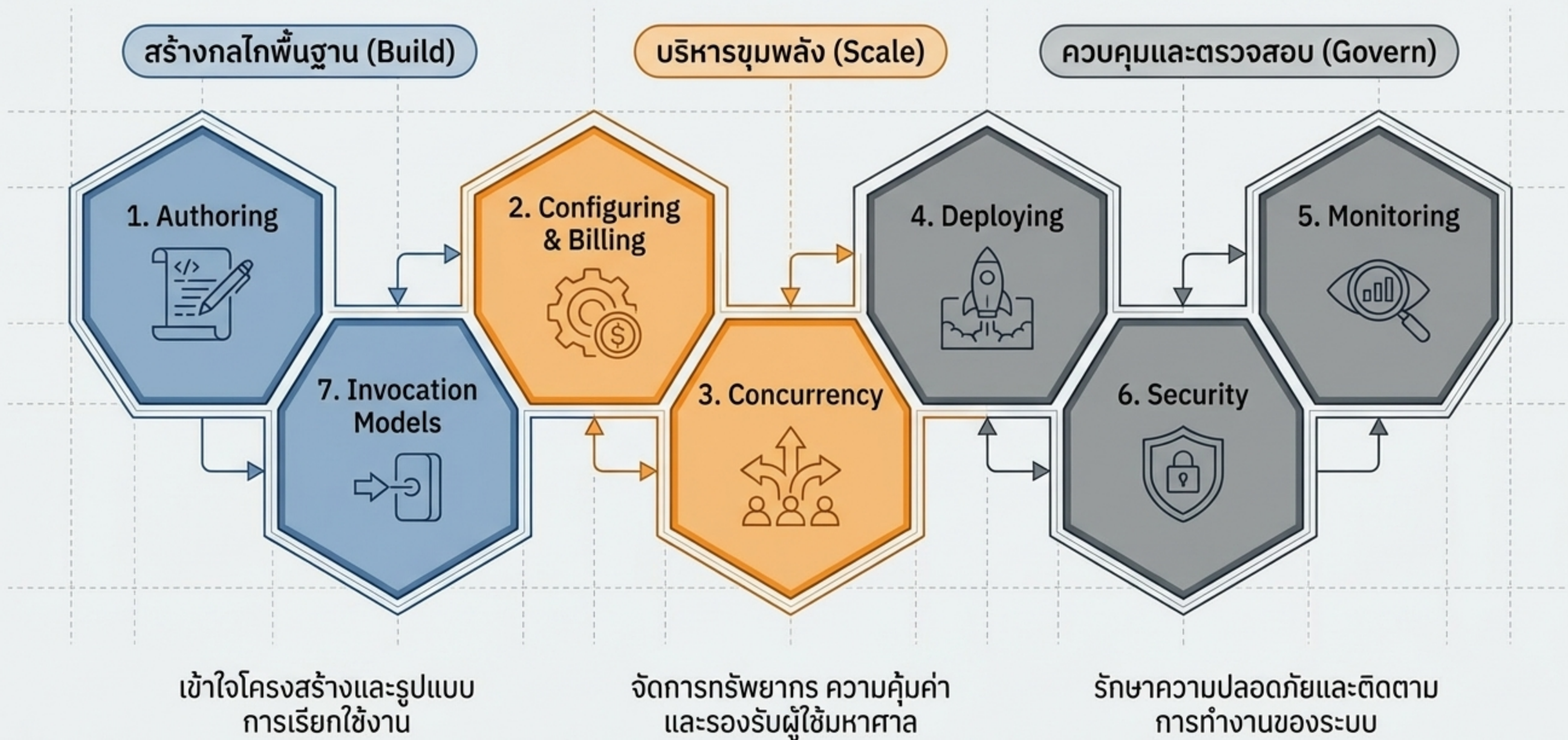
เปรียบเสมือนการซื้อบ้านสำเร็จรูป
คุณต้องจ่ายค่าบำรุงรักษาและเปิดไฟทิ้งไว้ตลอดเวลาแม้ไม่มีใครอยู่



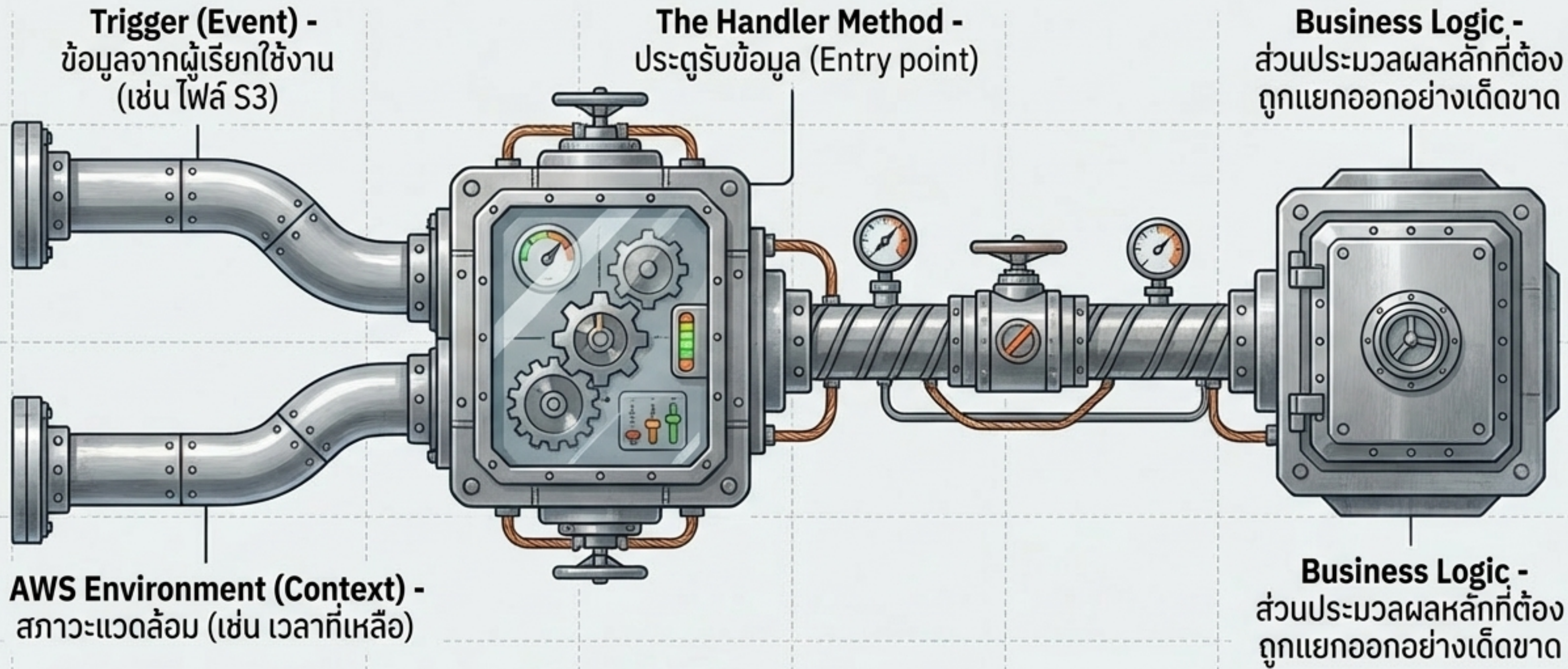
Serverless (AWS Lambda)

เปรียบเสมือนการวาดพิมพ์เขียว (Infrastructure as Code - IaC)
ระบบจะถูกสร้างขึ้นมากันทันทีเมื่อมีความต้องการใช้งาน
และสลายไปเมื่อทำงานเสร็จสิ้น

7 เสาหลักสู่การเป็นผู้เชี่ยวชาญสถาปัตยกรรม AWS Lambda

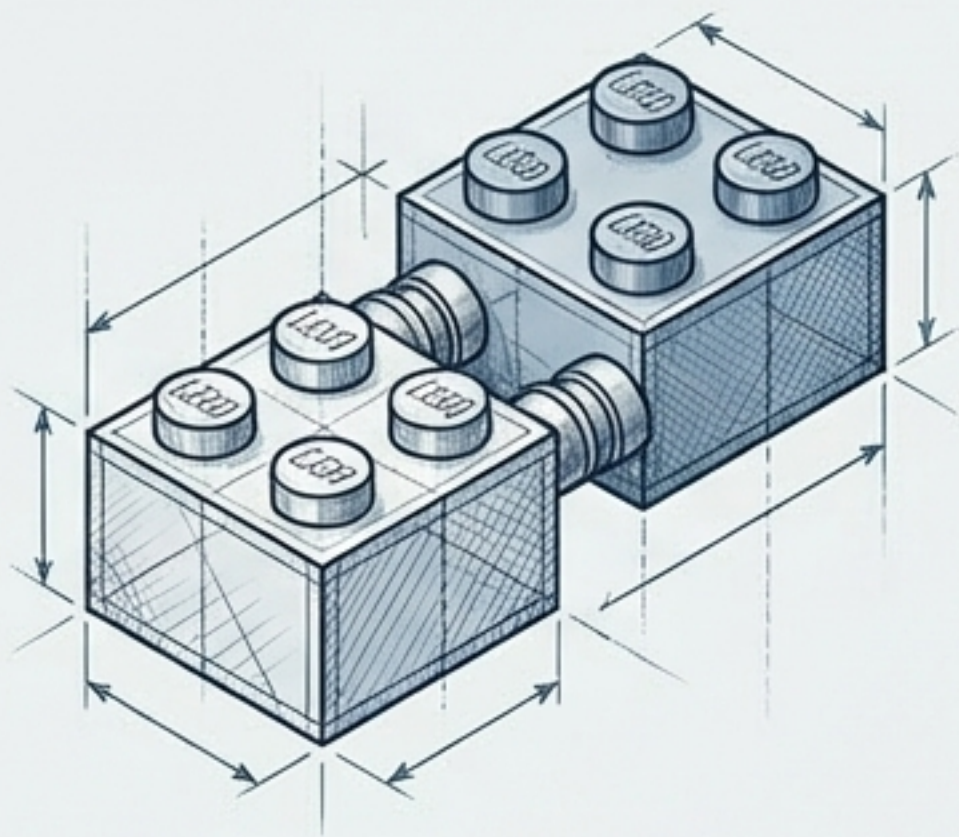


โครงสร้างการทำงานภายในและการแยกส่วนประมวลผล



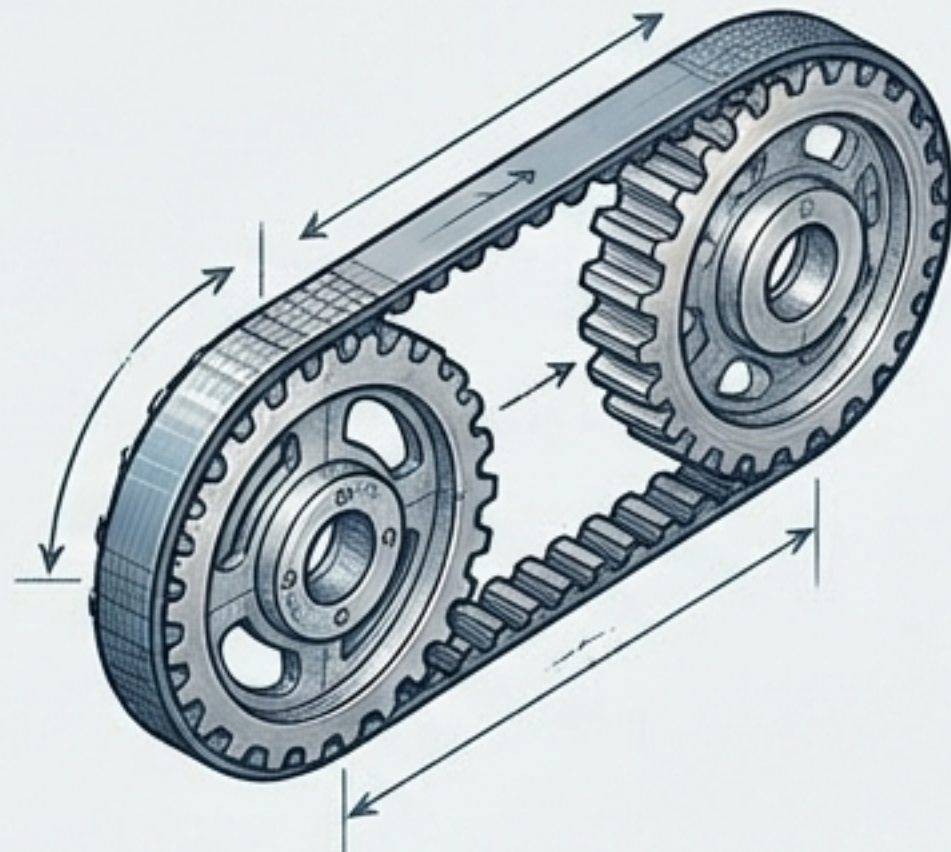
รองรับภาษาหลากหลาย: Node.js, Python, Java, Go, C#, Ruby, PowerShell และ Custom Runtimes

หลักการออกแบบฟังก์ชันเพื่อความยืดหยุ่นและการทดสอบที่แม่นยำ



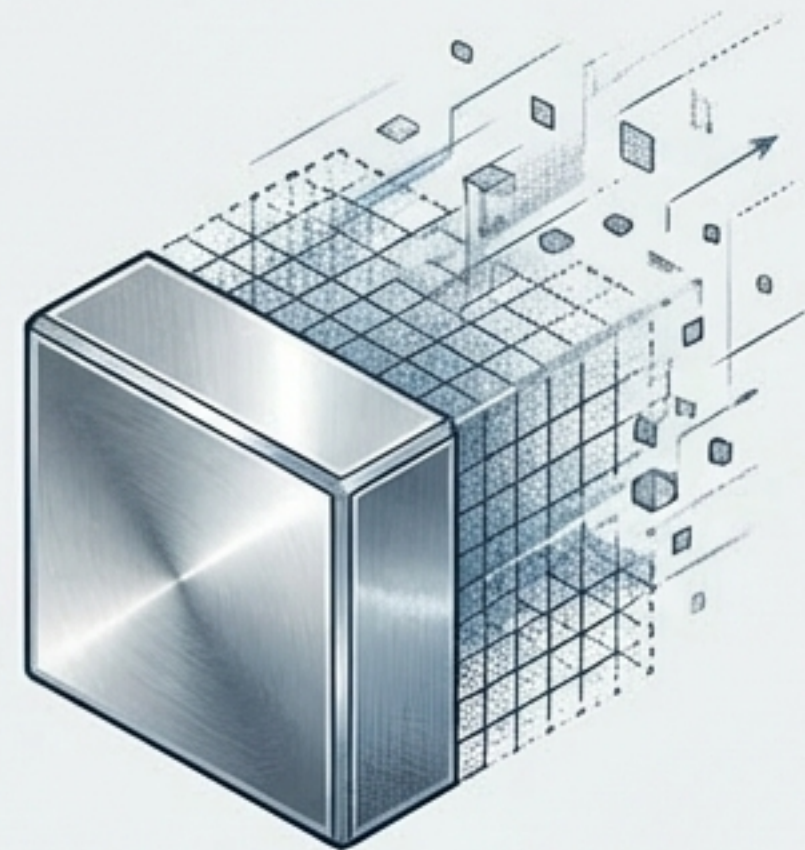
Modular Design (1 ฟังก์ชัน = 1 หน้าที่)

ออกแบบให้ฟังก์ชันทำหน้าที่
เดียวอย่างสมบูรณ์
เช่น แยกฟังก์ชันบีบอัดรูปภาพ
ออกจากฟังก์ชันดึงข้อมูล



Separate Business Logic

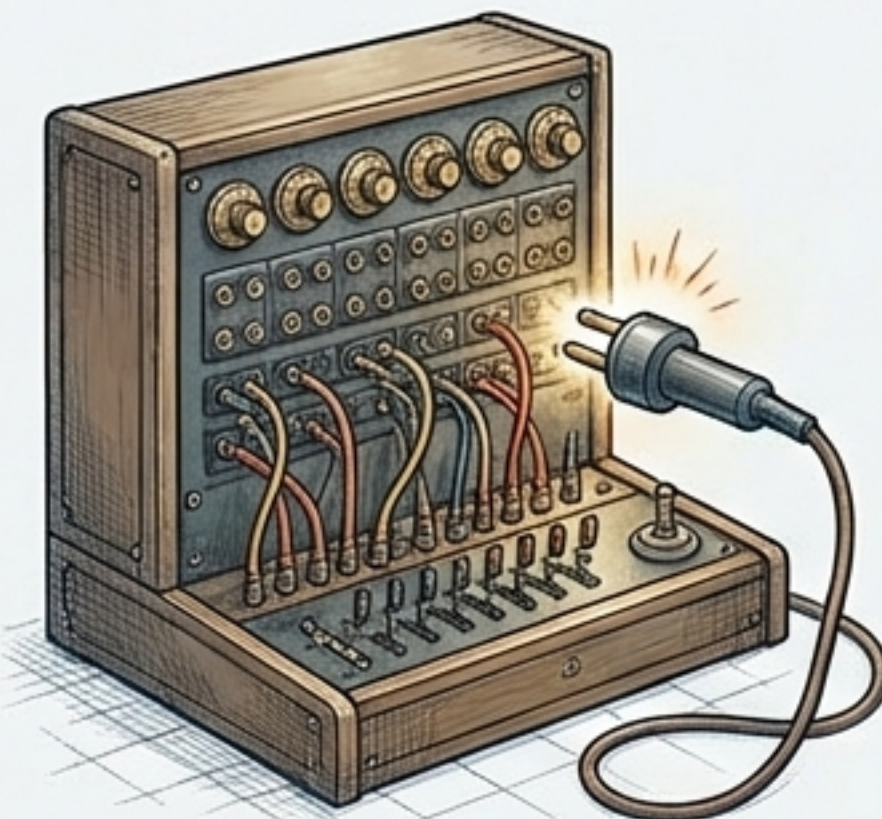
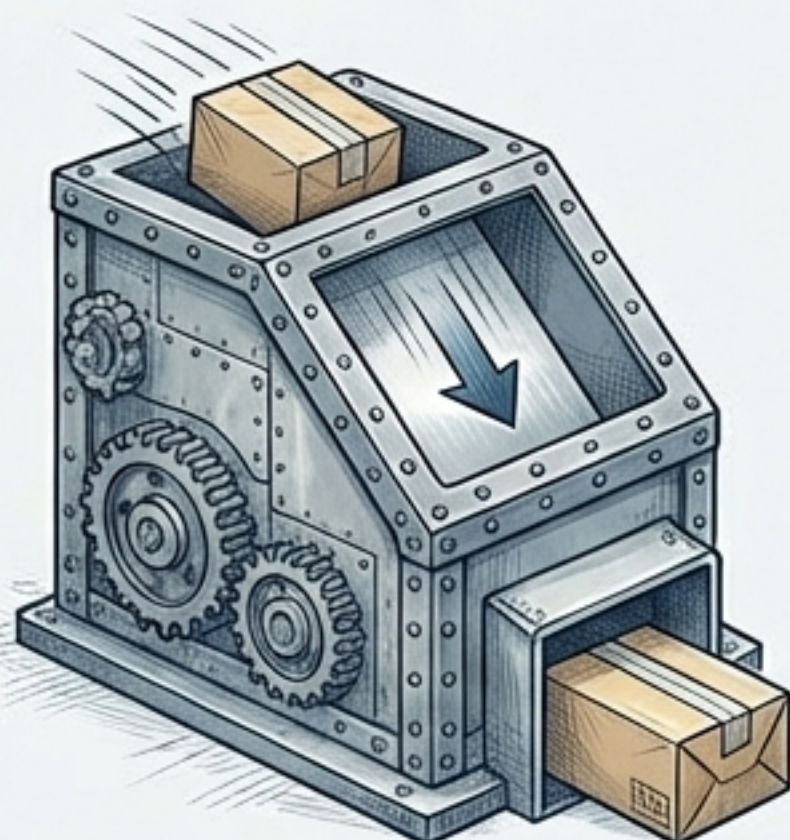
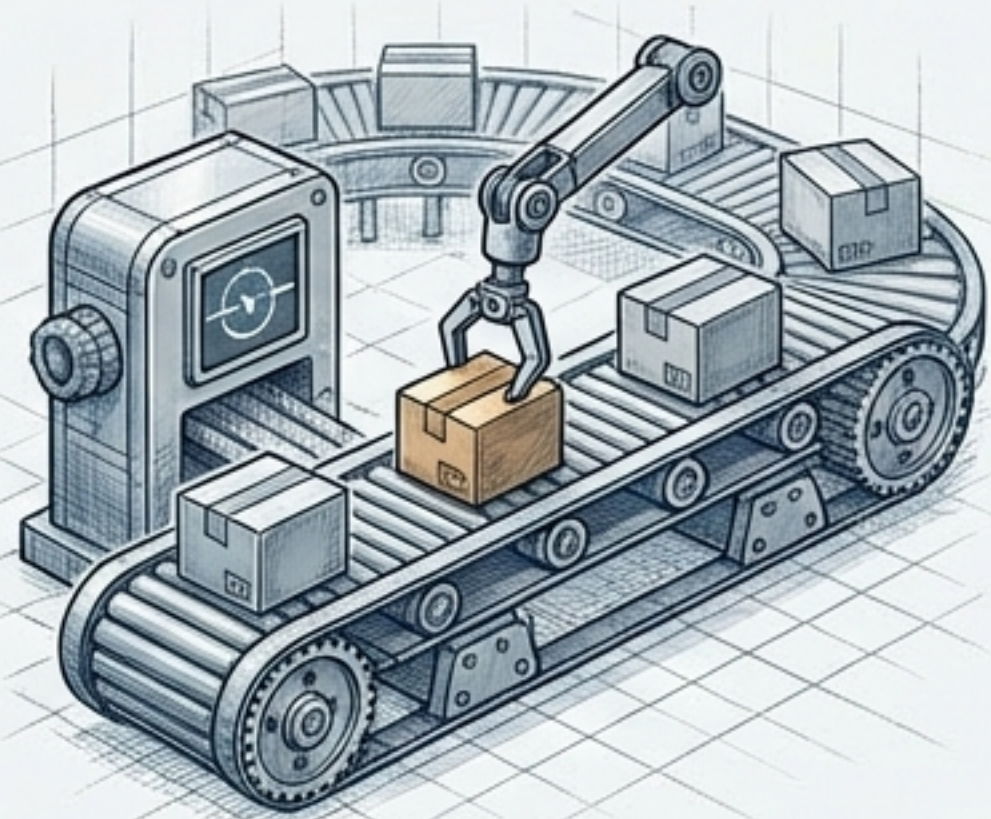
การแยกโค้ดประมวลผลออกจาก
Handler ช่วยให้สามารถทำ Unit Test
ได้ง่ายและนำโค้ดไปใช้ซ้ำได้



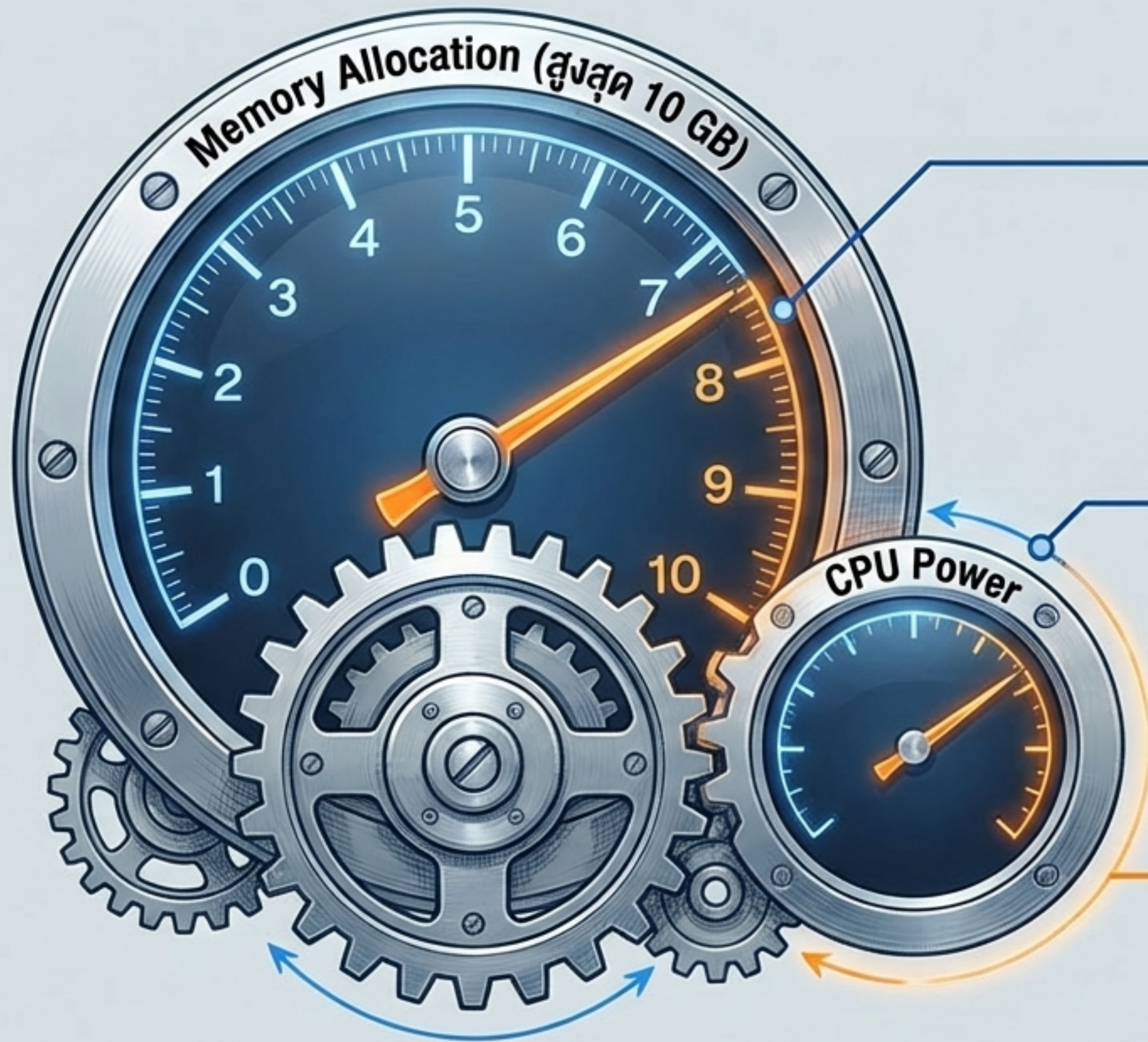
Statelessness (ไร้สถานะ)

ฟังก์ชันจะดำรงอยู่เมื่อถูกเรียกใช้เท่านั้น
ห้ามบันทึกข้อมูลสถานะใดๆ ไว้ใน
Execution Environment
(ทุกการเรียกใช้คือการเริ่มต้นใหม่)

3 รูปแบบการเรียกใช้งานที่ตอบสนองความต้องการต่างกัน

	Synchronous	Asynchronous	Polling
			
กลไก:	ส่งงานแล้วรอคำตอบกลับทันที	ส่งงานแล้วลืม (Fire and forget)	ดึงข้อมูลมาประมวลผลอย่างต่อเนื่อง
ลักษณะ:	ระบบจะหยุดรอนกว่าฟังก์ชันจะทำงานเสร็จ	ส่งข้อมูลเข้าระบบแล้วไปทำงานอื่นต่อได้เลย ไม่ต้องรอผลลัพธ์	Lambda เป็นผู้ดึงข้อมูลจาก Stream หรือ Queue ที่ละชุด
ตัวอย่าง:	Amazon API Gateway	Amazon S3 Events	Amazon SQS, DynamoDB Streams

จ่ายตามทรัพยากรที่จัดสรรด้วยหน่วยวัดระดับมิลลิวินาที



การจัดสรรทรัพยากร:

สามารถกำหนด Memory ได้สูงสุด 10 GB (ระบบจะจัดสรร CPU และเครื่อง่ายให้แปรผันตาม Memory โดยอัตโนมัติ)

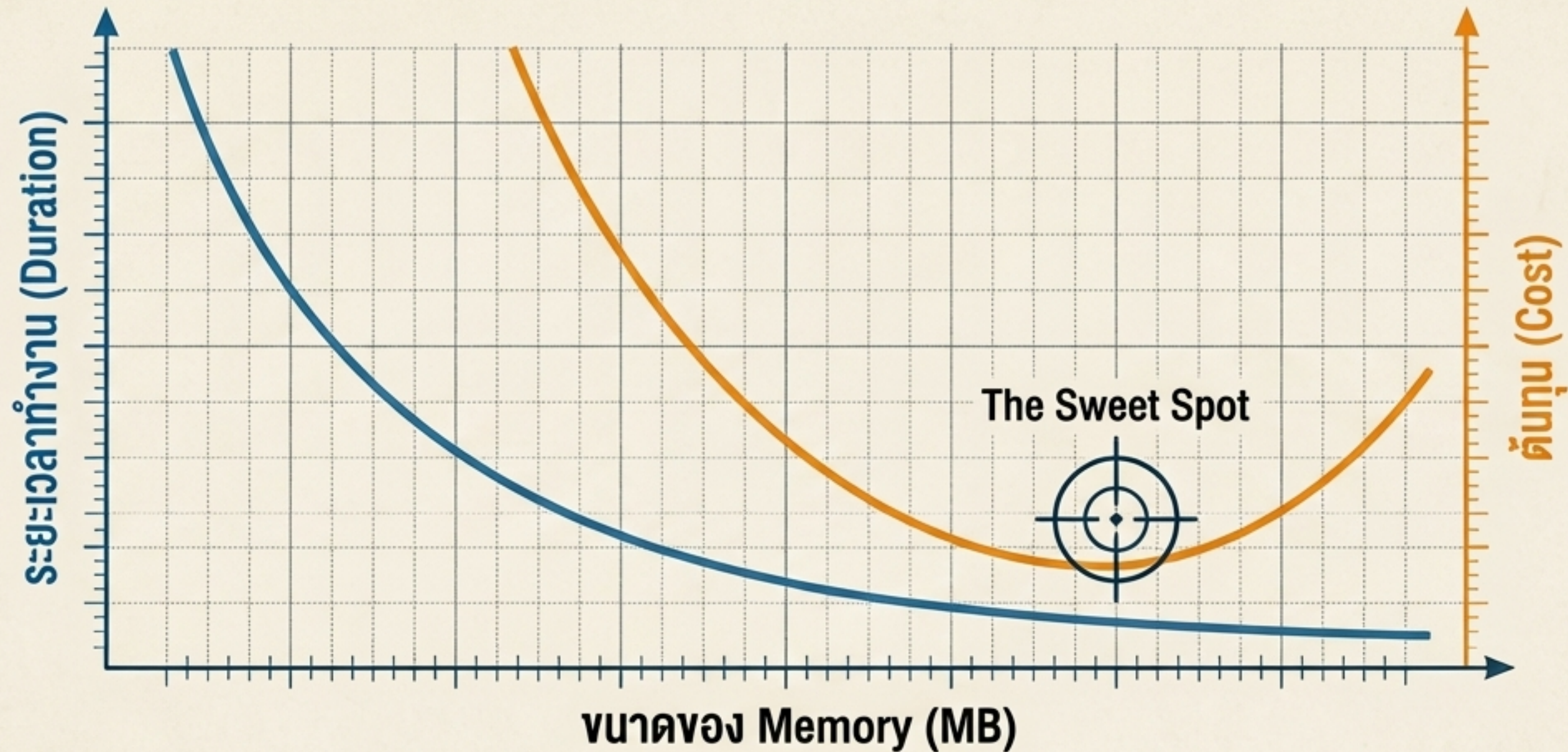
รูปแบบการคิดเงิน (Billing):

จ่ายตาม 'ทรัพยากรที่ตั้งค่าไว้ (Allocated)' ไม่ใช่ตามที่ 'ใช้จริง (Used)' โดยคิดรอบละ 1 มิลลิวินาที

กลยุทธ์ Timeout (Fail Fast):

ตั้งเวลา Timeout ได้สูงสุด 15 นาที แต่ควรตั้งค่าให้สั้นที่สุดหลังจากการทดสอบโหลด เพื่อป้องกันระบบค้าง และเกิดค่าใช้จ่ายบานปลาย

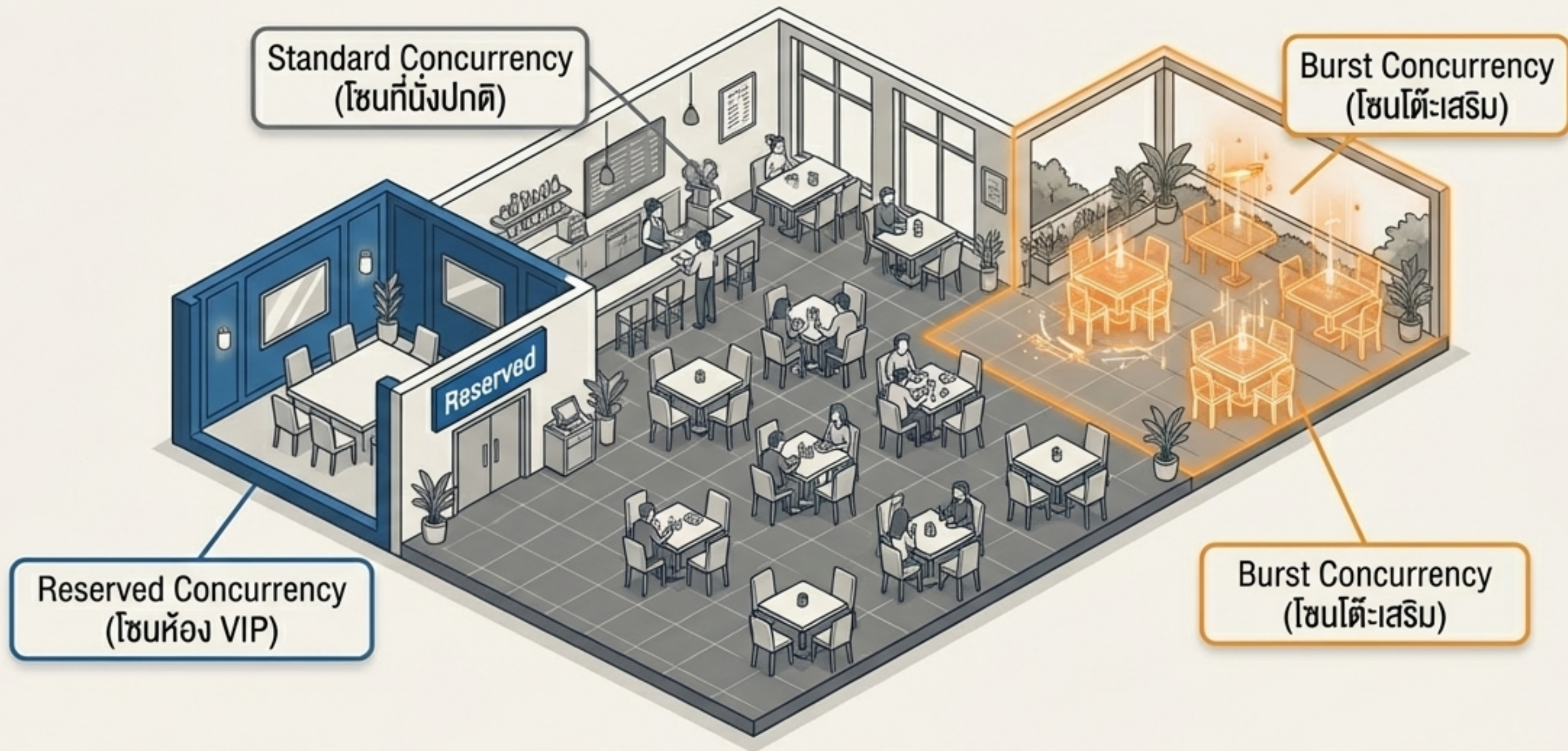
ปรากฏการณ์ Power Tuning: เมื่อการเพิ่มสเปคช่วยลดต้นทุน



ความขัดแย้งเชิงต้นทุน (The Paradox):

- การตั้งค่า Memory ต่ำสุดไม่ได้แปลว่าประหยัดที่สุดเสมอไป
 - การเพิ่ม Memory = เพิ่มความแรง CPU = ฟังก์ชันทำงานเสร็จเร็วขึ้นอย่างก้าวกระโดด
 - เมื่อระยะเวลาทำงานลดลงฮวบฮาบ ส่งผลให้ ค่าใช้จ่ายรวมต่อครั้งลดลง แม้จะใช้ Memory สูงขึ้นก็ตาม
- คำแนะนำ: ใช้เครื่องมือ AWS Lambda Power Tuning เพื่อค้นหาจุดสมดุลที่สมบูรณ์แบบนี้โดยอัตโนมัติ

Concurrency: กลไกการรองรับผู้ใช้งานพร้อมกันจำนวนมาก

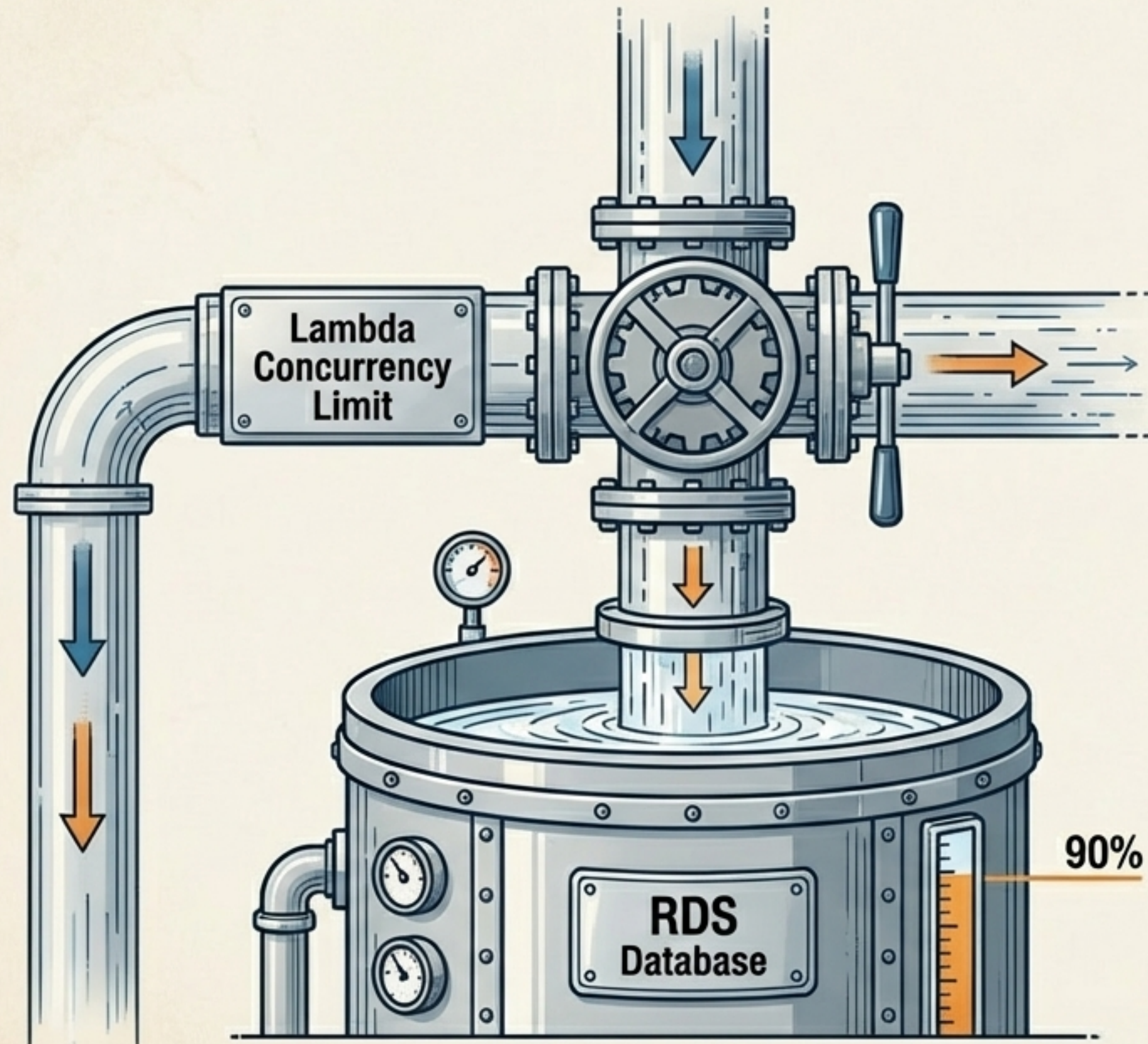


Concurrency คือ 'ความจุที่นั่ง' ของระบบในเสี้ยววินาทีนั้นๆ

Burst: รองรับกราฟ/พีคพุ่งกะทันหัน (เช่น 3,000 requests) และขยายเพิ่ม 1,000 อินสแตนซ์ ทุกๆ 10 วินาที

Reserved: การันตีทรัพยากรไว้ให้ ฟังก์ชันที่สำคัญที่สุด ป้องกันไม่ให้ถูกแย่งโควตาจนหมด

การจำกัดวงเบต (Rate Limiting) เพื่อปกป้องสถาปัตยกรรมรวม



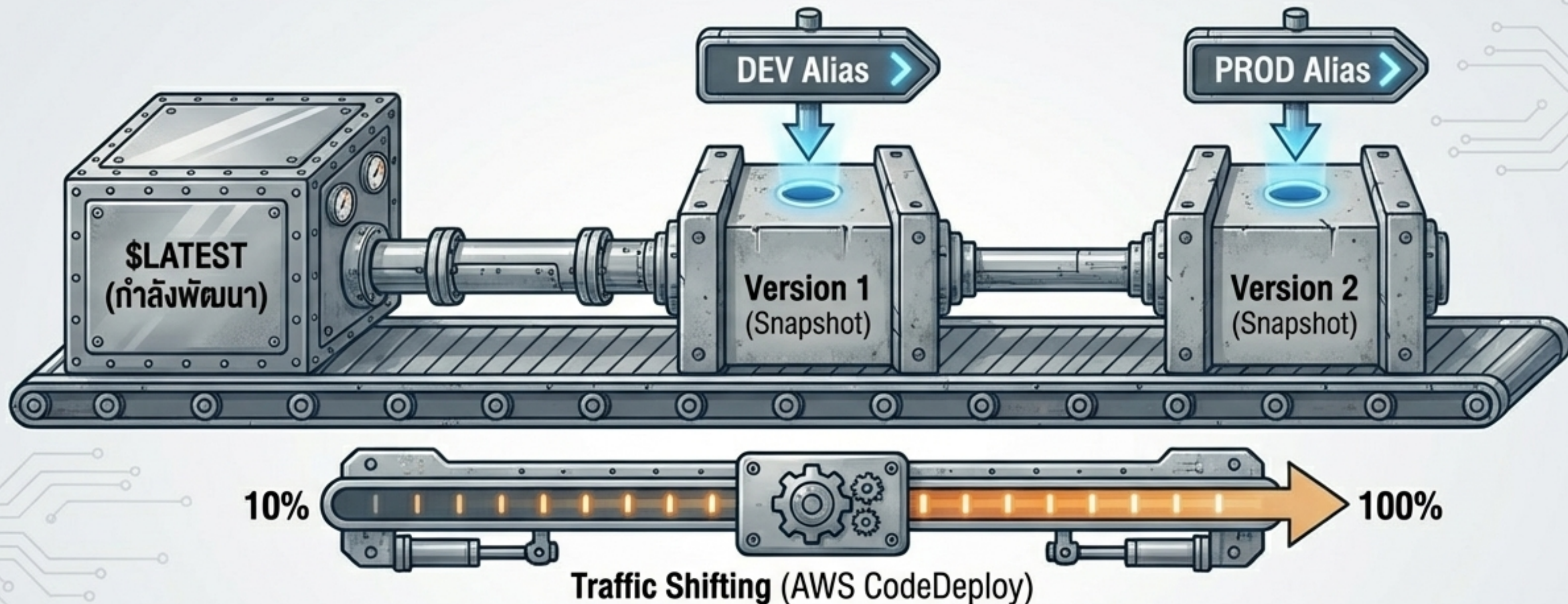
ทำไม Lambda ที่ขยายตัวได้ไร้ขีดจำกัด
จึงต้องถูกจำกัดความเร็ว?

1. ป้องกันคอขวดของระบบปลายทาง:
ป้องกันไม่ให้ Lambda สาด Request
จำนวนมหาศาลจนเกินขีดจำกัดของ
Database Connection (เช่น RDS ล่ม)

2. ควบคุมงบประมาณ:
ป้องกันค่าใช้จ่ายที่อาจพุ่งสูงอย่าง
ควบคุมไม่ได้จากเหตุการณ์ที่ไม่คาดคิด

3. จัดระเบียบ Batch Processing:
ควบคุมจังหวะเวลาในการประมวลผล
ข้อมูลปริมาณมหาศาลให้ราบรื่นสม่ำเสมอ

การปรับใช้ระบบใหม่และการย้ายผู้ใช้งานอย่างปลอดภัย



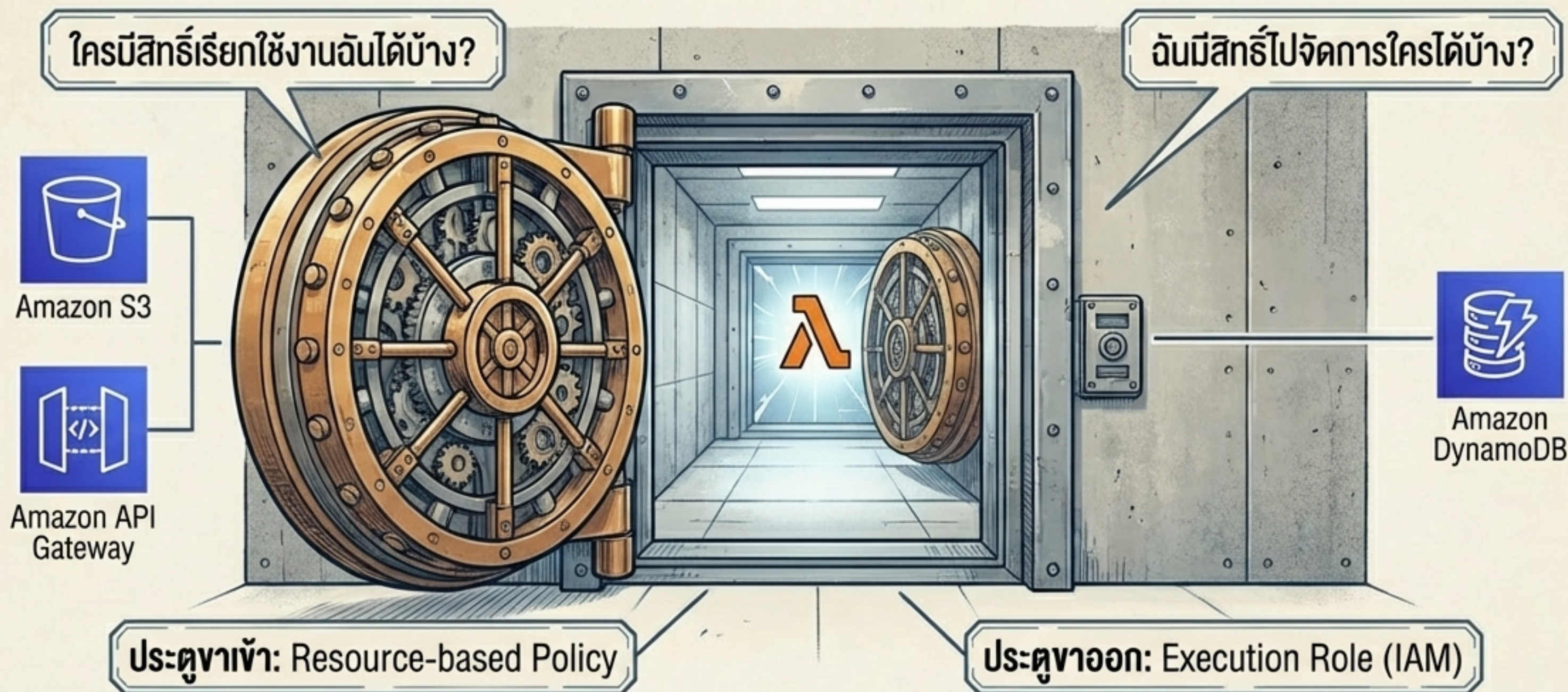
Versions & Aliases:

- สร้าง Snapshot ของโค้ดที่ไม่สามารถแก้ไขได้ (Immutable) และใช้ Alias เป็นเสมือน 'ป้ายบอกทาง' ไปยังเวอร์ชันที่ต้องการ
- **AWS SAM**: เครื่องมือสร้าง Blueprint แบบย่อที่แปลงคำสั่งสั้นๆ ให้กลายเป็น AWS CloudFormation ขึ้นสูง

Safe Deployments:

- ลดความเสี่ยงในการขึ้นระบบใหม่ด้วยการทยอยย้ายผู้ใช้งาน (Traffic Shifting) แบบค่อยเป็นค่อยไป (Canary หรือ Linear)

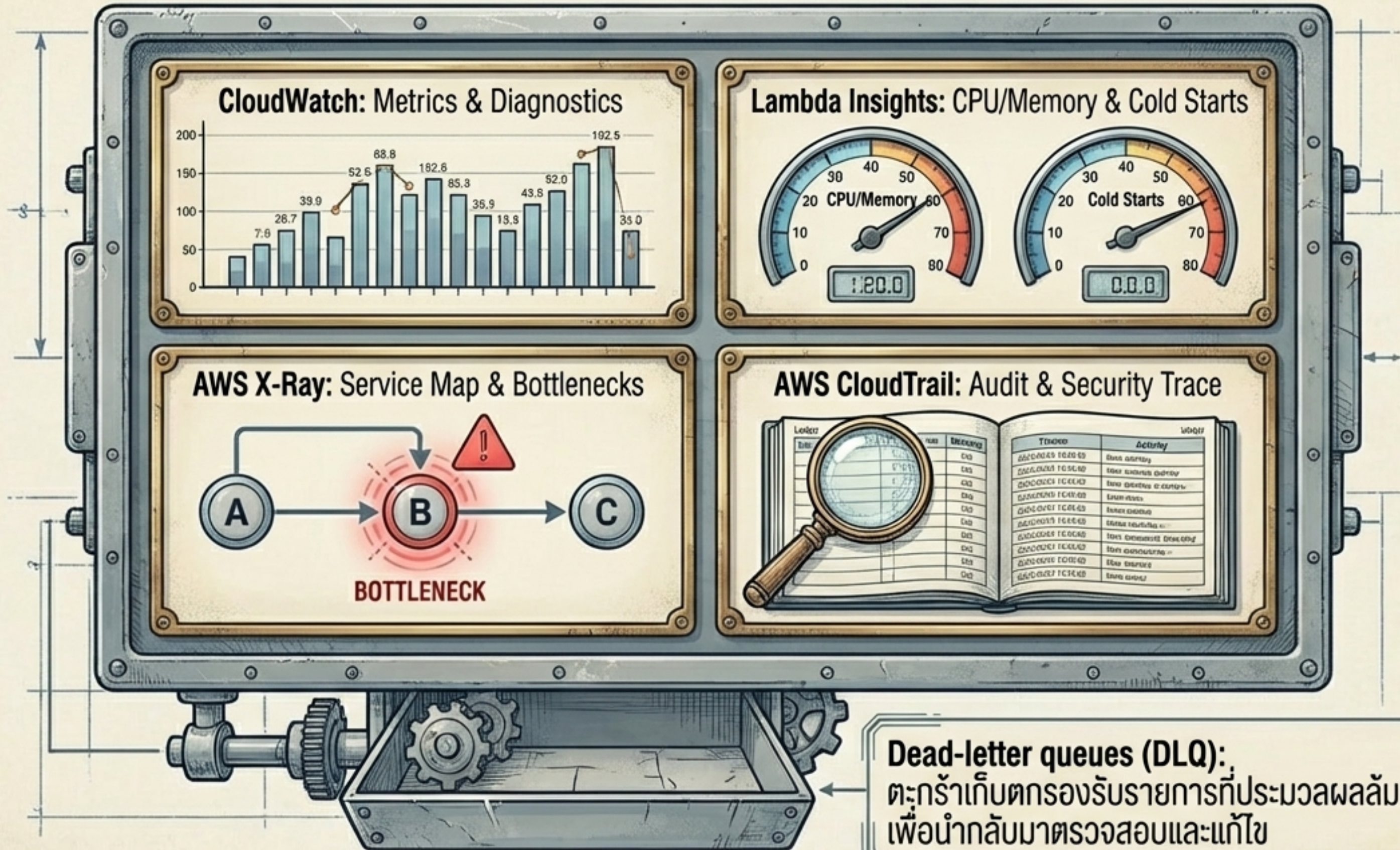
ระบบรักษาความปลอดภัย 2 ชั้น (Dual-Layer Security)



การตั้งค่าสิทธิ์ (IAM) ต้องครอบคลุมทั้ง ‘ผู้เรียก’ และ ‘ผู้ถูกเรียก’

- **Resource-based Policy:** กำหนดว่าใคร หรือ บริการใด (เช่น S3) ที่ได้รับอนุญาตให้ส่งคำสั่ง Invoke ฟังก์ชันนี้
- **Execution Role:** กำหนดว่าตัวฟังก์ชันนี้ มีสิทธิ์ที่จะไปกระทำการใดๆ กับบริการอื่นของ AWS บ้าง (เช่น เขียนข้อมูลลง Database)

ระบบตรวจสอบและติดตามสถานะแบบ 360 องศา



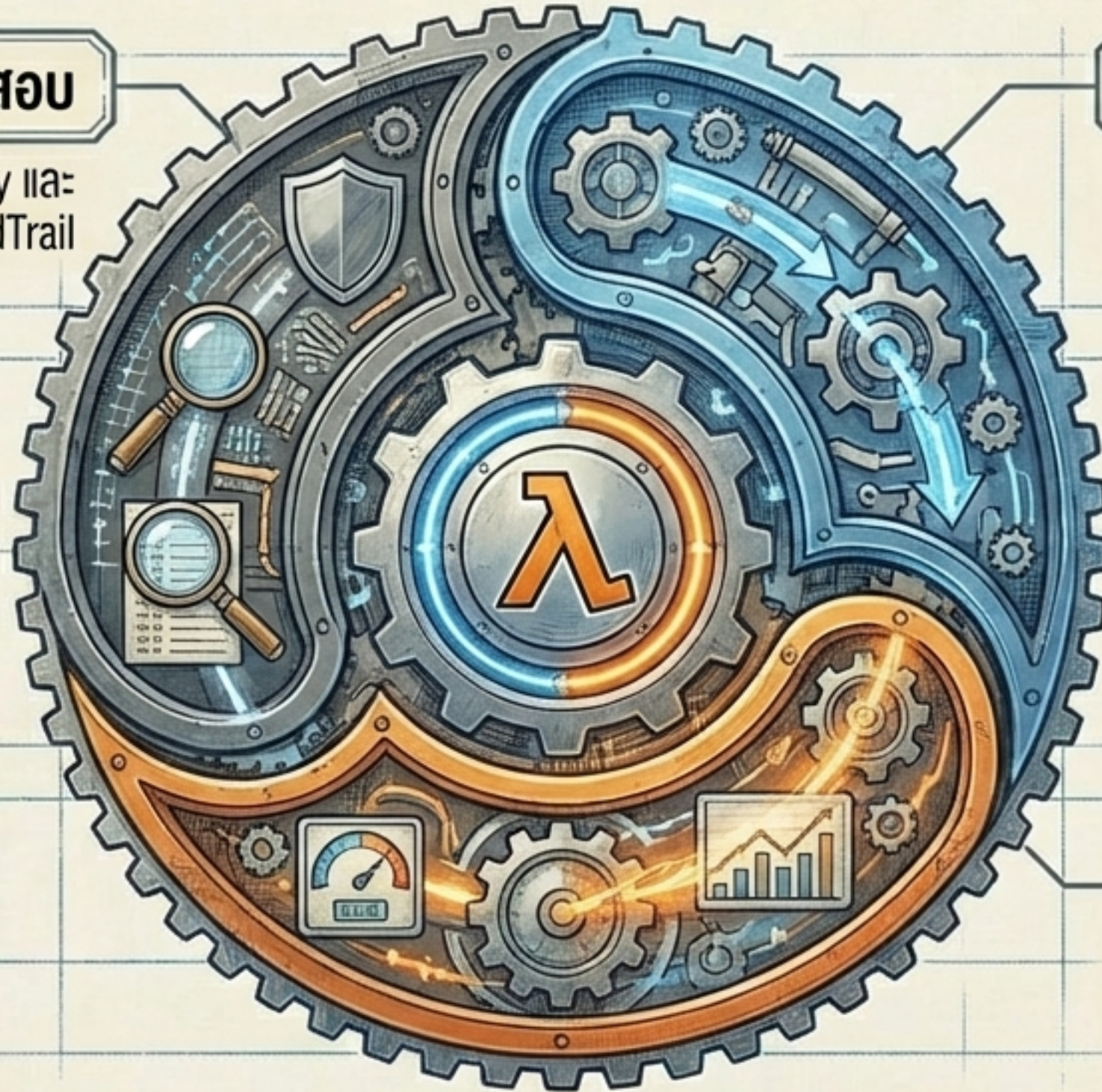
วงจรแห่งสถาปัตยกรรม Serverless ที่สมบูรณ์แบบ

GOVERN: ควบคุมและตรวจสอบ

ป้องกันระบบด้วย Dual-Layer Security และ
ตรวจสอบย้อนหลังด้วย X-Ray & CloudTrail

BUILD: สร้างกลไก

ออกแบบฟังก์ชันแบบ Modular
ไร้สถานะ และใช้ SAM สร้าง Blueprint



SCALE: บริหารขุมพลัง

หาจุดคุ้มทุนด้วย Power Tuning
และจัดระเบียบด้วย Concurrency Limit

ระบบที่รวดเร็ว คุ่มค่า และปลอดภัย เกิดจากการประสานกันของ 7 เสาหลัก
ที่ขับเคลื่อนสถาปัตยกรรมแห่งการขยายตัวให้เติบโตได้อย่างยั่งยืนและไร้ขีดจำกัด